K.R. MANGALAM UNIVERSITY

SCHOOL OF ENGINEERING &TECHNOLOGY

Department of Computer Science & Technology

Assignment Number: 1

Unit Title: Foundations & Algorithmic Analysis

Assignment Title: Algorithmic Efficiency & Recursion Toolkit

Course Code: ETCCDS202

Course Name: Data Structures

Faculty: Mrs. Neetu Chauhan

Student Name: Shreya Verma

Roll Number: 2501730173

Section: A

Programme: B. Tech CSE (Specialization in AI & ML)

PART-A

What is an ADT?

An Abstract Data Type (ADT) is a logical model of a data structure that defines what operations can be performed and what those operations do, without specifying how they are implemented internally. It focuses on behavior, not implementation. For example, a Stack ADT defines push and pop operations but does not specify whether it uses an array or a linked list.

Why are stack operations usually O(1)?

Stack operations like push, pop, and peek are performed only at the top of the stack. They do not require traversal or comparison of elements. Therefore, the time taken does not depend on the number of elements in the stack, making their time complexity constant — O(1) in best, average, and worst cases.

PART-B

Factorial (Recursive)

Time Complexity

$$T(n) = T(n-1) + O(1)$$

Each call makes one recursive call and one multiplication. Total number of calls = n

- Big-O: $O(n)$
- Big-Ω: $\Omega(n)$
- Big-Θ: $\Theta(n)$

Space Complexity

- Recursive depth = n
- $O(n)$, $\Theta(n)$

Fibonacci (Naive Recursive)

Time Complexity

$T(n) = T(n-1) + T(n-2) + O(1)$

Each call creates two more calls, forming a recursion tree.
Number of calls grows exponentially.

- **Big-O:** $O(2^n)$
- **Big-Ω:** $\Omega(2^n)$
- **Big-Θ:** $\Theta(2^n)$

Space Complexity

Maximum recursion depth = n

- **$O(n)$, $\Theta(n)$**

Fibonacci (Memorized)

Each Fibonacci number is calculated only once and stored.

Time Complexity

Total computations = n

- **Big-O:** $O(n)$
- **Big-Ω:** $\Omega(n)$
- **Big-Θ:** $\Theta(n)$

Space Complexity

Memo storage + recursion stack

- **O(n), Θ(n)**

Why Naive Fibonacci is Slow for Large n?

- It recomputes the same subproblems many times.
- Number of recursive calls grows exponentially.
- Running time increases very fast as n increases.
- Hence, it becomes inefficient for large n.

PART-C

Why number of moves becomes $2^n - 1$?

To move n disks:

- First, move (n−1) disks to auxiliary rod
- Then move the largest disk (1 move)
- Then move (n−1) disks to destination So recurrence is:

  $T(n) = 2T(n-1) + 1$

  Solving this recurrence gives:

  $T(n) = 2^n - 1$

  That is why the number of moves grows exponentially.

Time Complexity

Since number of moves = $2^n - 1$,

Time Complexity = $O(2^n)$

The algorithm must print every move, so time grows exponentially as n increases.

Space Complexity

Recursion depth = n
At maximum, n recursive calls are on the call stack at once.
So:
Space Complexity = **O(n)**

PART-D

Binary search divides the array into two halves at each step and continues searching in only one half.

Recurrence Relation
$T(n) = T(n/2) + O(1)$

- $T(n/2) \rightarrow$ search in half of the array
- $O(1) \rightarrow$ constant time comparison After each step, problem size becomes half:
  $n \rightarrow n/2 \rightarrow n/4 \rightarrow n/8 \rightarrow \ldots \rightarrow 1$

  Number of times we can divide n by 2 until it becomes 1 is:
  $\log_2(n)$
  Therefore:
- **Time Complexity = O(log n)**
- **Big-Θ = Θ(log n)**
- **Big-Ω = Ω(1)** (best case)

PART-E

Binary Search – Divide & Conquer

Binary Search follows the Divide & Conquer paradigm because it repeatedly divides the problem into smaller subproblems.

In each step:

1. The array is divided into two halves.
2. The middle element is checked.
3. Only one half is selected for further searching.

So the original problem of size n becomes a smaller problem of size n/2. This process continues until the element is found or the size becomes 1. It is called Divide & Conquer because:

- Divide: Split the array into two halves.
- Conquer: Recursively search in one half.
- Combine: Return the result (no complex combining needed).

This reduces the time complexity from linear O(n) (in linear search) to logarithmic O(log n), making it highly efficient.

Fibonacci (Memoized) – Dynamic Programming

Memoized Fibonacci follows the **Dynamic Programming** paradigm.
Dynamic Programming is used when:

- A problem has **overlapping subproblems**
- The same smaller problems are solved multiple times

In naive Fibonacci, values like fib(3) or fib(4) are calculated repeatedly. This causes exponential growth in computation.
Memoization solves this problem by:

- Storing already computed Fibonacci values.
- Checking the stored result before recomputing. So each Fibonacci number is calculated only once.

Thus, memoization removes repeated work and reduces time complexity from **$O(2^n)$** to **$O(n)$**.

This is why memoized Fibonacci is an example of Dynamic Programming.

Tower of Hanoi – Recursion

Tower of Hanoi is a classic example of recursion because it solves a big problem by solving smaller versions of the same problem.

To move n disks:

1. Move $n-1$ disks to auxiliary rod.
2. Move the largest disk.
3. Move $n-1$ disks to destination rod.

Here, the problem of moving n disks is reduced to moving $n-1$ disks.

Base Case:

If $n = 1$, move the disk directly from source to destination.

Without the base case, recursion would continue infinitely. The base case ensures that recursion stops.

So Tower of Hanoi clearly shows:

- Problem reduced into smaller subproblem.
- Same logic applied repeatedly.
- Stopping condition defined.

Real-Life Examples of Paradigms

Divide & Conquer (Real-Life Example)

Searching for a word in a dictionary.

Instead of checking every page one by one:

- You open the middle.
- Decide whether the word lies before or after.
- Then repeat on that half.

This is similar to Binary Search.

Dynamic Programming (Real-Life Example)

Saving frequently used calculations in a calculator.

If you already solved a difficult math problem earlier and wrote it down, you don't solve it again. You reuse the stored result.

This is similar to memoization in Fibonacci.

Greedy Algorithm (Real-Life Example)

Giving change using coins.

If you need to give ₹47 and coins available are 10, 5, 2, 1:

- First take largest coin (10),
- Then next largest,
- Continue until total is reached.

At each step, you make the best immediate choice without thinking about future steps. This is a Greedy approach.

Conclusion

Different algorithm paradigms use different strategies:

- **Divide & Conquer** reduces the problem size step by step.
- **Dynamic Programming** avoids repeated work by storing solutions.
- **Recursion** solves problems using smaller subproblems of the same type.
- **Greedy** makes the best immediate choice at each step.

Understanding these paradigms helps in choosing the most efficient approach for solving computational problems.